VISUAL NOTES • COMPLETE GUIDE

Visual Notes — Claude Code Subagents

Claude Code Subagents

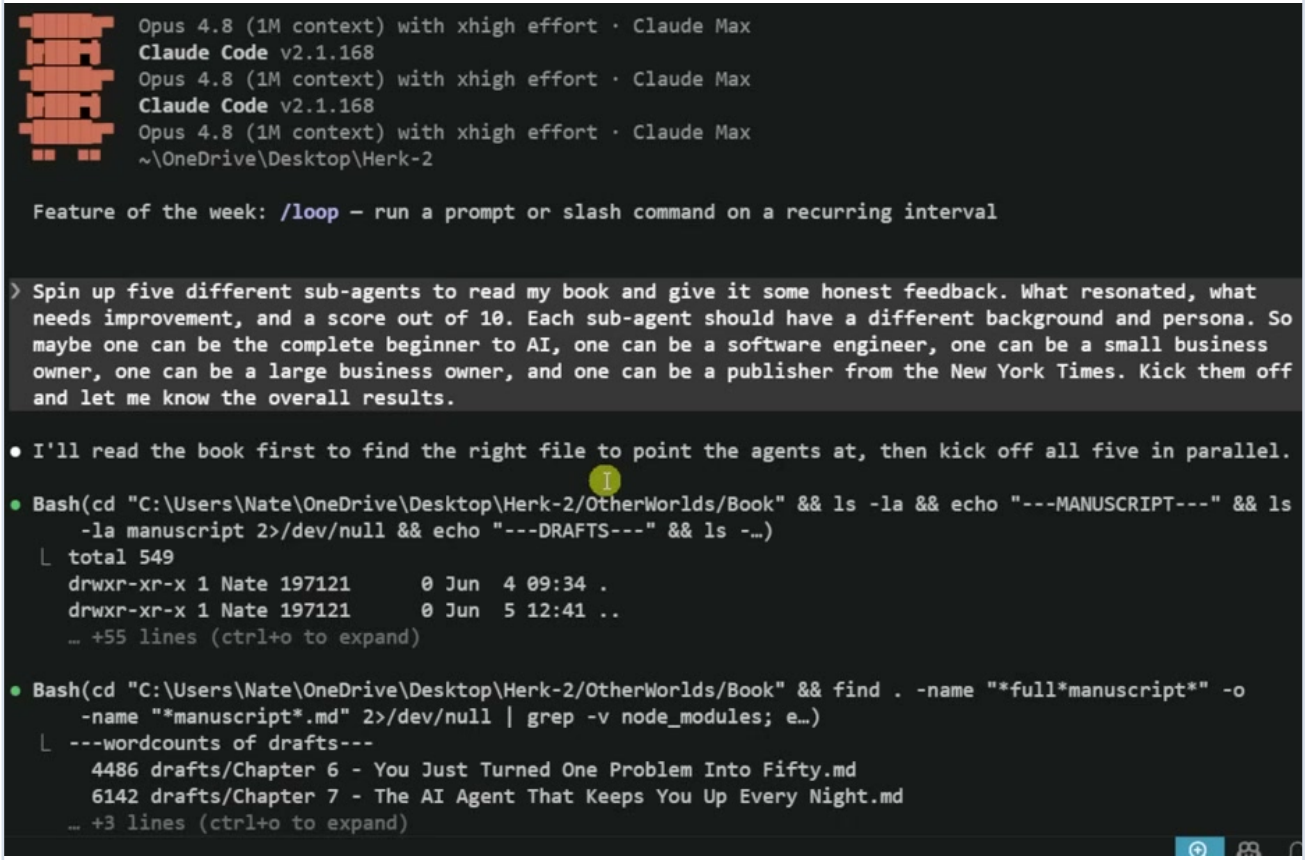
26:42

- 30 substantive visual(s), selected by content
- Repeated builds, presenter-only shots, and filler excluded
- Output length expands or contracts with the source material
- Claims, uncertainty, and caveats are labeled clearly

https://www.youtube.com/watch?v=e18sdZLwP7o

One Main Session, Five Parallel Perspectives

Independent personas review the same material at the same time.



NOTES

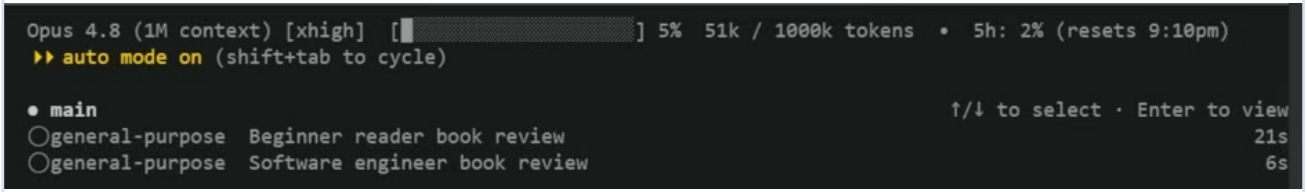
- A subagent is an independent Claude Code task session created by the main session to perform delegated work and return a result.
- The main context is the conversation history and working information held by the primary session; delegation keeps most worker detail outside it.
- Five reviewers can receive the same source material while applying distinct beginner, engineering, business, publishing, or enterprise lenses.
- Because the reviews are independent, they can run in parallel and be synthesized after completion rather than executed one after another.

CORE IDEA

Parallel subagents turn one request into several independent viewpoints without filling the main context with every intermediate step.

Inspect Each Delegated Session

The interface exposes the worker, prompt, task, and live status.



NOTES

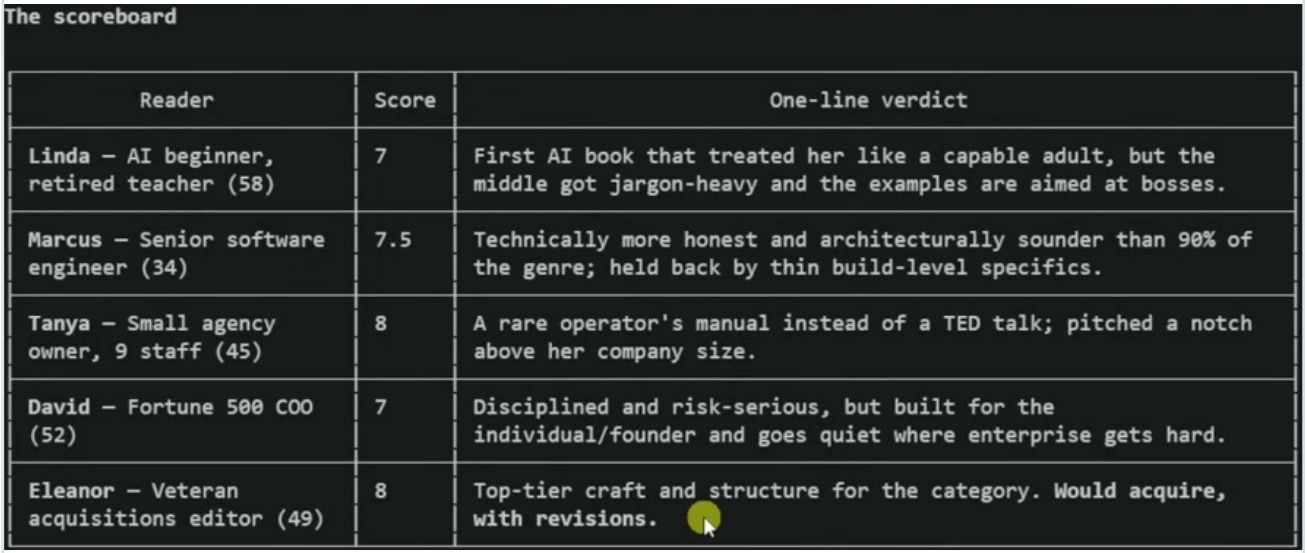
- The session list separates the main session from each active worker, making parallel execution visible.
- Opening a worker reveals the exact prompt that the main session supplied, including its assigned persona and task.
- A worker can use a different model, expertise profile, and instruction set while still reporting only to the main session.
- This inspection path helps verify that delegation matched the intended role instead of silently relying on a generic prompt.

CORE IDEA

Inspectable worker prompts make parallel delegation debuggable rather than opaque.

The Main Session Orchestrates the Workers

Delegated research, file reading, and bug work return as compact reports.



The scoreboard

Reader	Score	One-line verdict
Linda — AI beginner, retired teacher (58)	7	First AI book that treated her like a capable adult, but the middle got jargon-heavy and the examples are aimed at bosses.
Marcus — Senior software engineer (34)	7.5	Technically more honest and architecturally sounder than 90% of the genre; held back by thin build-level specifics.
Tanya — Small agency owner, 9 staff (45)	8	A rare operator's manual instead of a TED talk; pitched a notch above her company size.
David — Fortune 500 COO (52)	7	Disciplined and risk-serious, but built for the individual/founder and goes quiet where enterprise gets hard.
Eleanor — Veteran acquisitions editor (49)	8	Top-tier craft and structure for the category. Would acquire, with revisions.

NOTES

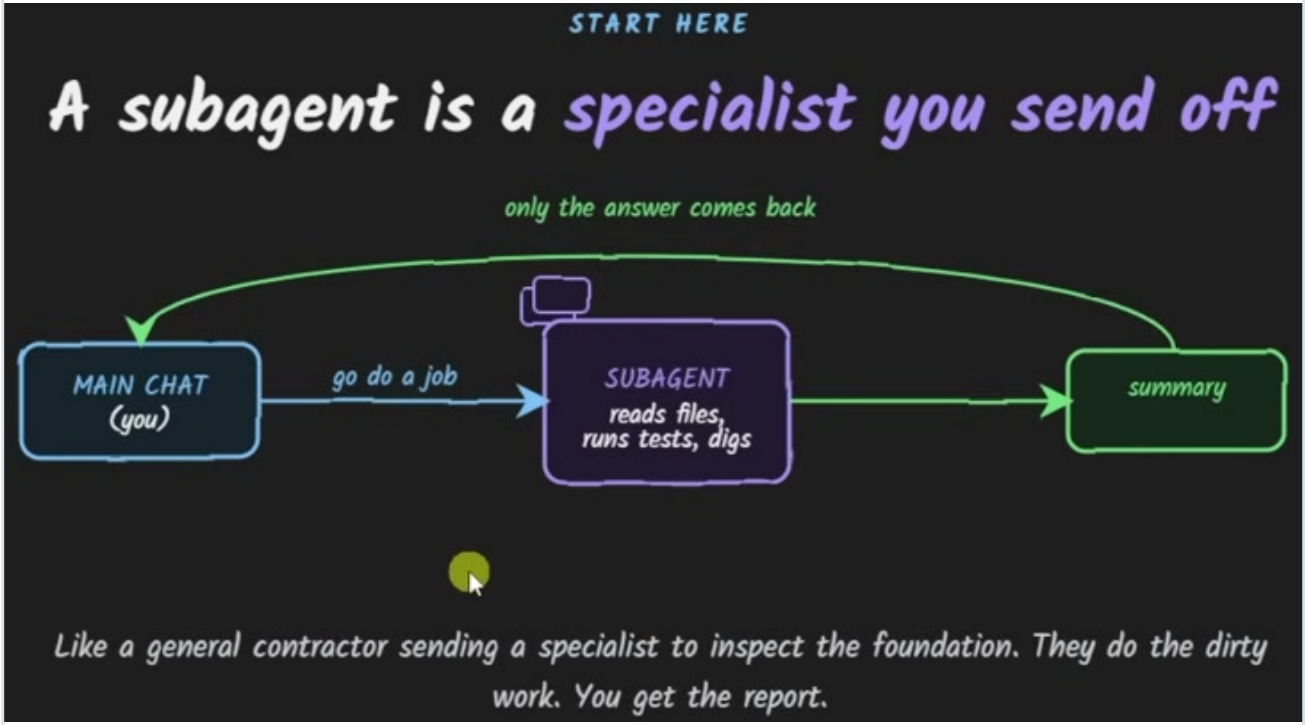
- The main session communicates with the user, assigns bounded tasks, and combines the returned findings.
- Workers communicate back to the main session rather than directly coordinating with one another.
- Useful assignments include reading many files, researching a topic, fixing an isolated bug, or producing an independent review.
- The returned report can be much smaller than the worker's full search and reasoning trail.

CORE IDEA

A hub-and-spoke structure lets one session coordinate specialists while retaining a compact decision record.

Context Isolation Protects the Primary Task

Fresh worker sessions absorb high-volume research and return only useful output.



NOTES

- Long chats accumulate files, search results, and discarded reasoning that can crowd the primary session's finite context window.
- A delegated worker starts with a fresh context and receives only the prompt and resources required for its assignment.
- For example, a research worker can collect features, operation details, and pricing without placing every web result in the primary conversation.
- Model names, context indicators, limits, and prices shown in the interface are version-sensitive and should be checked against current documentation.

CORE IDEA

Isolation preserves the primary session for decisions while disposable research detail stays in a separate context.

Built-In Workers Differ from Custom Agents

Generic workers accept ad hoc prompts; custom agents persist as files.



NOTES

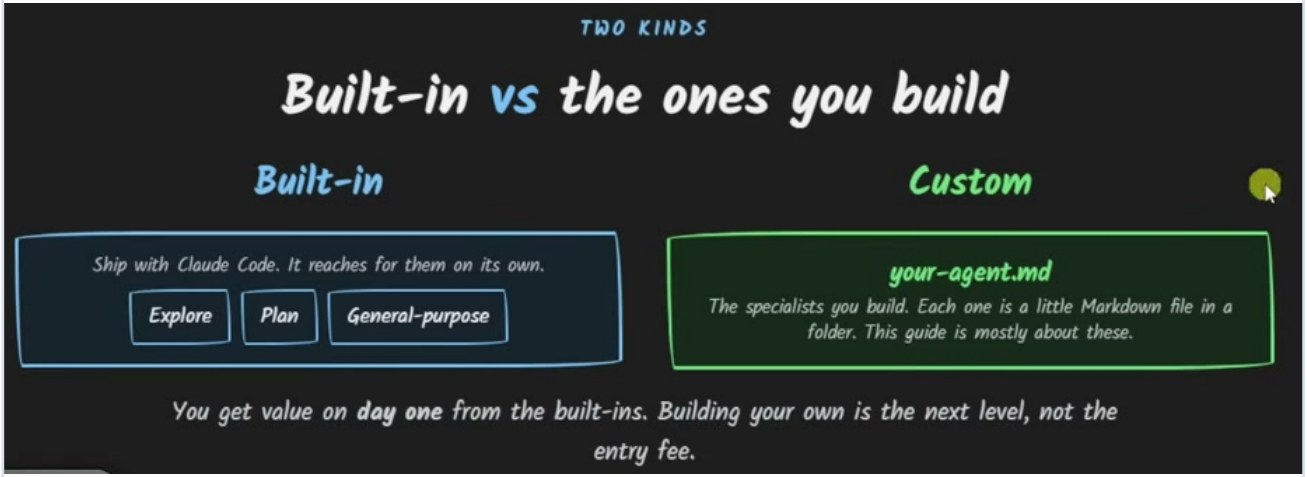
- Built-in workers such as general-purpose or research agents may be selected automatically for ordinary delegation.
- Giving a built-in worker a persona does not create a reusable custom agent; it only changes that invocation's prompt.
- A custom agent is stored as a Markdown file under a Claude agents directory, so its configuration and operating procedure can be reused and shared.
- A custom file contains YAML front matter—a structured metadata block between delimiter lines—followed by Markdown instructions.

CORE IDEA

Use built-ins for temporary delegation and custom files when the role, controls, and workflow must persist.

Agent Files Combine Metadata and Procedure

A compact configuration block controls discovery; the body controls execution.



NOTES

- The metadata block can declare fields such as `name`, `description`, `model`, `color`, allowed tools, disallowed tools, skills, and MCP access.
- The Markdown body explains the role, workflow, quality bar, output format, and any specialist knowledge the worker should apply.
- Progressive disclosure means reading compact metadata first and loading the full instructions only when the agent appears relevant.
- Available fields and accepted values are version-sensitive; have Claude Code consult current subagent documentation before generating configuration.

CORE IDEA

Separating discovery metadata from full instructions makes specialist behavior reusable without loading every procedure into every prompt.

The Description Is the Routing Rule

Precise task language improves automatic selection and reduces collisions.



NOTES

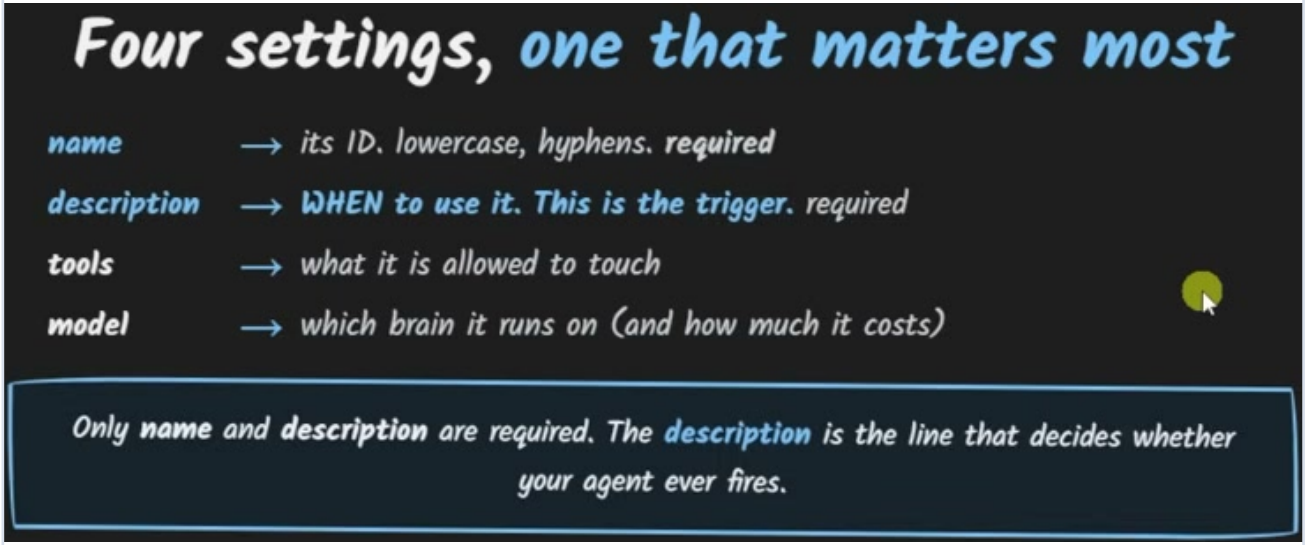
- Claude Code compares the current request with agent names and descriptions before deciding whether to load a full agent file.
- A useful description states the task, triggering situations, exclusions, and distinctive boundary from nearby skills or agents.
- Phrases such as `use proactively` may encourage broader selection, but trigger behavior is version-sensitive and must be tested rather than assumed.
- A misfire includes both failing to select the desired agent and selecting it for an unrelated request.

CORE IDEA

A precise description is both documentation and a practical router for dependable automatic invocation.

Front Matter Enforces Capability Boundaries

Tool and service access belong in configuration, not wishful prompting.



NOTES

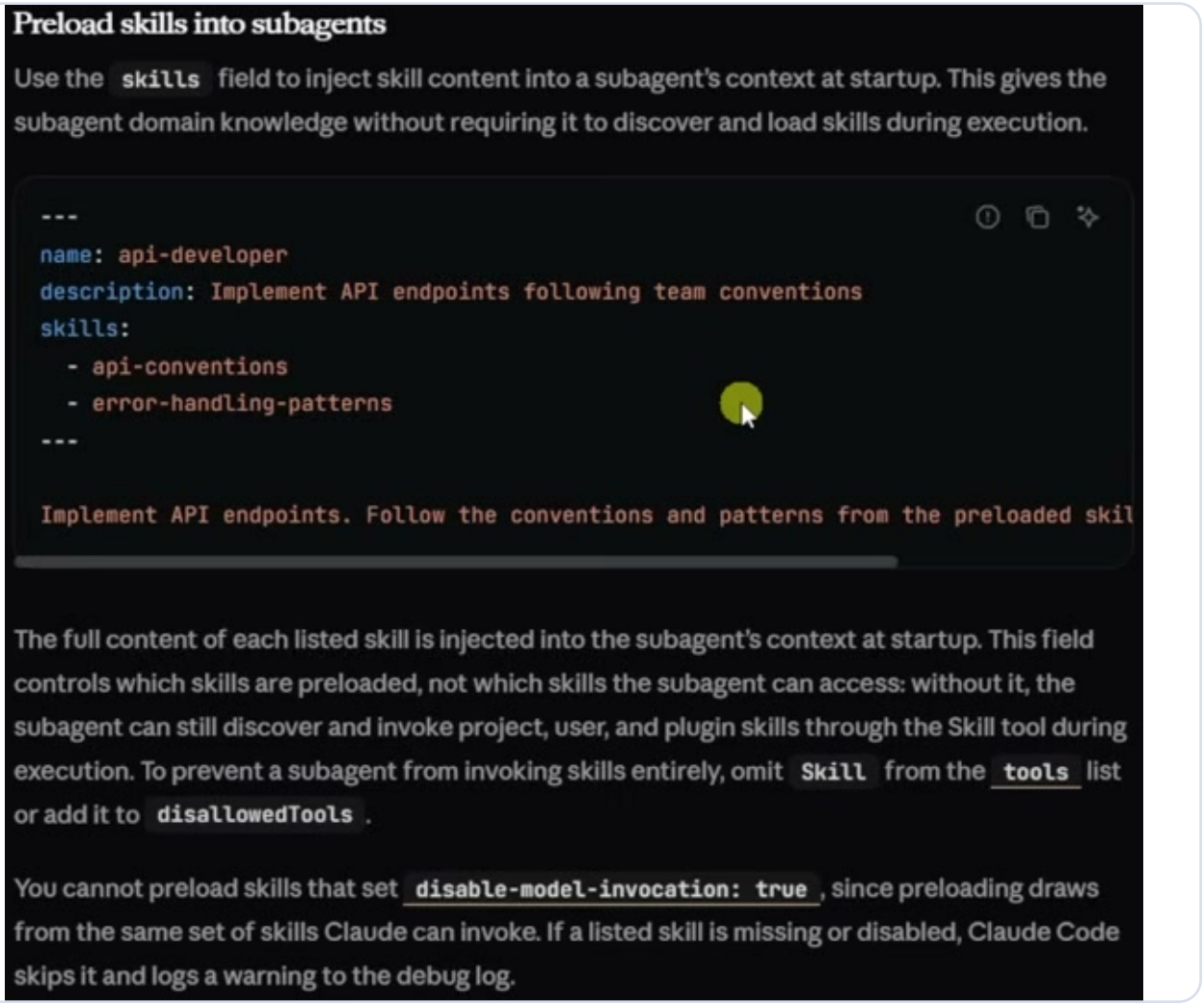
- Allowed and disallowed tool fields can make an agent read-only or restrict it to a narrow operational surface.
- MCP server access and reusable skills can also be granted selectively instead of inherited without review.
- Configuration should explicitly encode what data the worker may inspect and what actions it may take.
- Exact field names and tool identifiers can change; validate generated metadata with the current documentation and a safe test task.

CORE IDEA

Hard capability limits are stronger safeguards than telling an unconstrained agent not to perform an action.

Write the Body as an Operating Manual

Specialization comes from a clear sequence, standards, and feedback loops.



NOTES

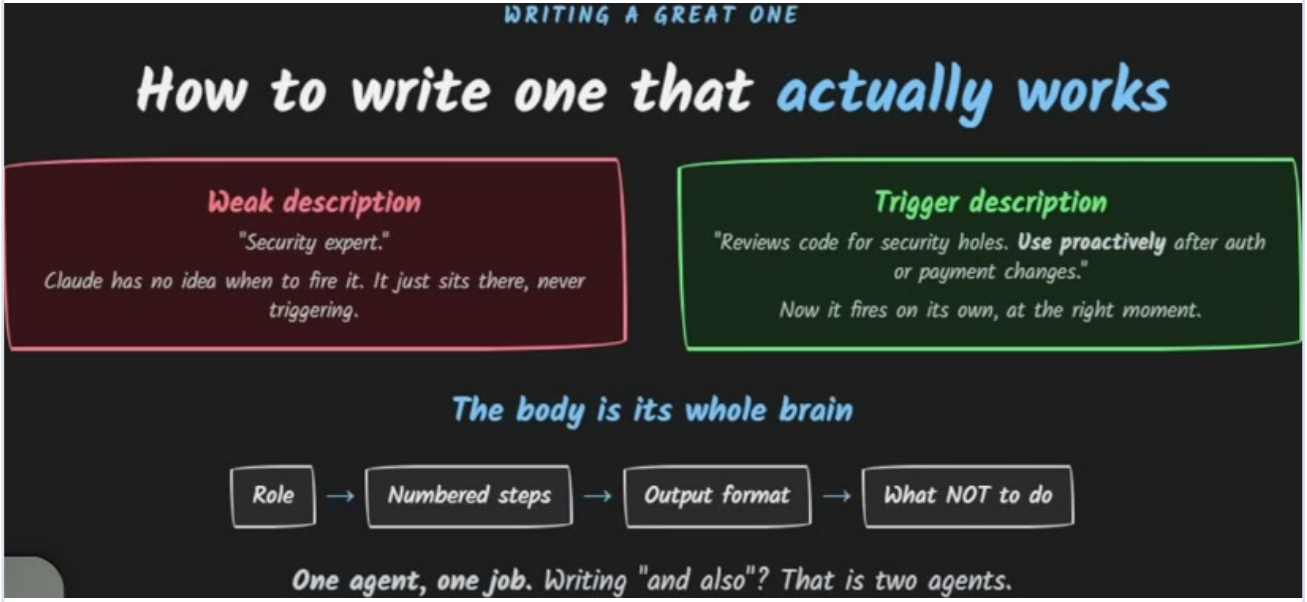
- After metadata routes the task, the body should define the ordered procedure the worker follows.
- Include the inputs to inspect, tools or skills to use, checks to perform, output structure, and stopping conditions.
- Subagents may invoke skills, and skills may delegate work to subagents; they are composable mechanisms rather than competing formats.
- Improve the file from observed failures and successes instead of expecting the first draft to cover every edge case.

CORE IDEA

Reliable specialization requires an explicit procedure and repeated refinement, not merely a job title.

Skills and Subagents Solve Different Context Problems

One enriches the current session; the other performs work in a separate one.



NOTES

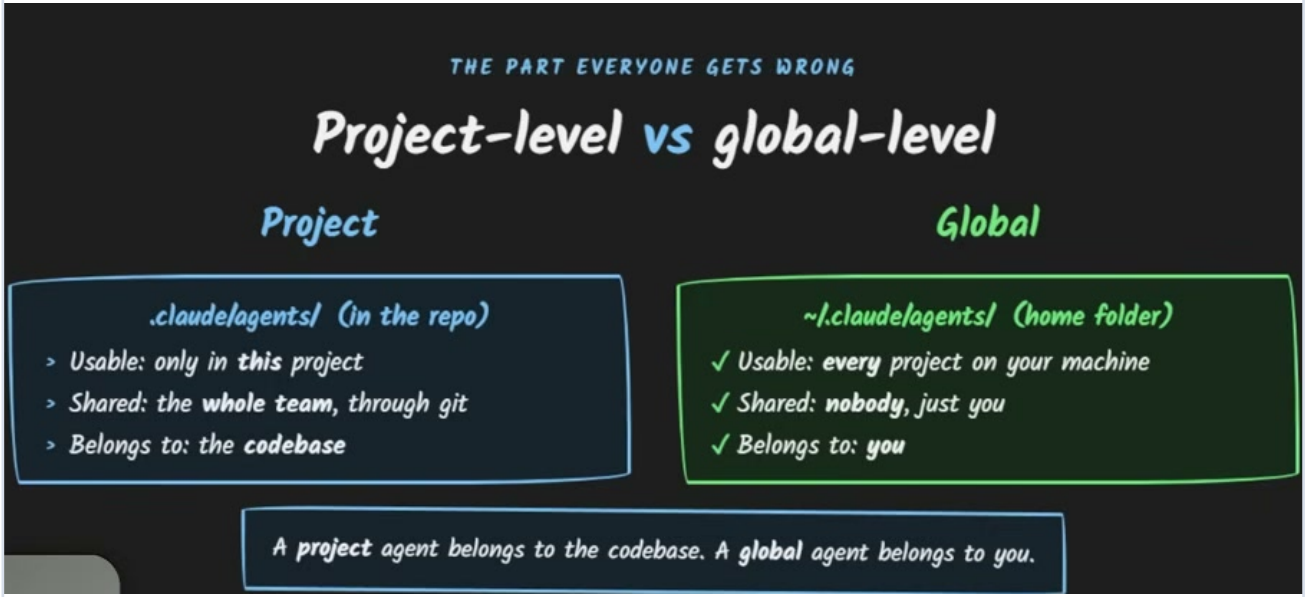
- A skill is reusable guidance normally loaded into the current session to improve how that session performs a task.
- A subagent executes in a separate context, can use a different model, and can run concurrently with other workers.
- Choose a skill when the primary session should retain the work; choose a subagent when isolation, parallelism, or a separate model is valuable.
- The mechanisms can be nested: a specialist worker may load a domain skill, while a skill may dispatch several independent workers.

CORE IDEA

Select the mechanism by context and coordination needs, not by treating skills and subagents as interchangeable labels.

Choose Project or Global Scope Deliberately

File location determines portability, ownership, and where an agent appears.



NOTES

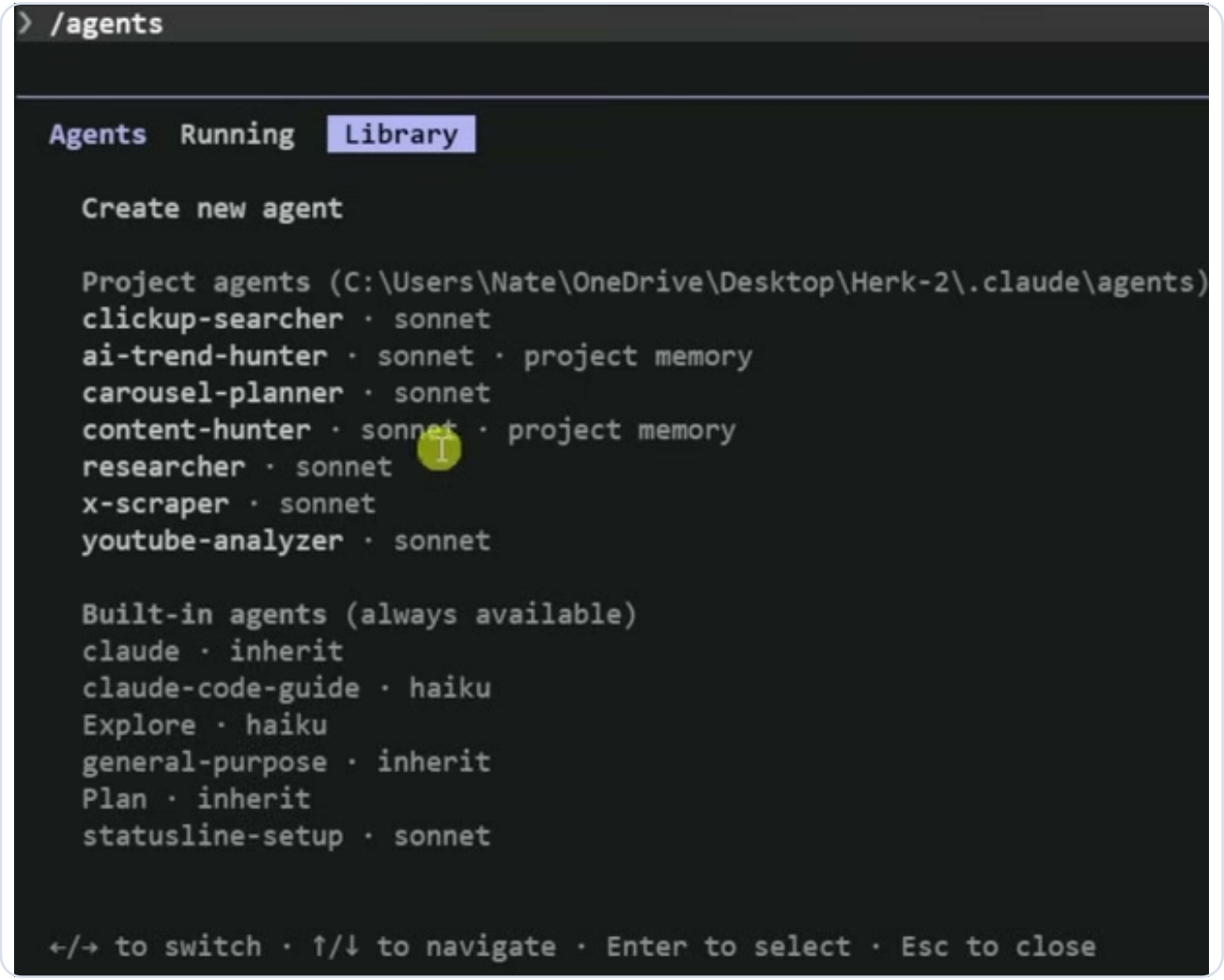
- Project scope means the agent file lives inside a repository's `.claude/agents/` directory and can travel with that repository.
- Global scope means the file lives in the user's Claude configuration and is available across projects on that machine.
- Store team workflows in the project; store personal, cross-project helpers globally, subject to current Claude Code path conventions.
- Because agents are Markdown files, moving or copying one between scopes is straightforward, but duplicated versions can drift.

CORE IDEA

Scope is a distribution decision: project agents belong to the work, while global agents belong to the user.

Open the Agent Manager with `/agents`

The manager lists running, built-in, project, and personal agents.



NOTES

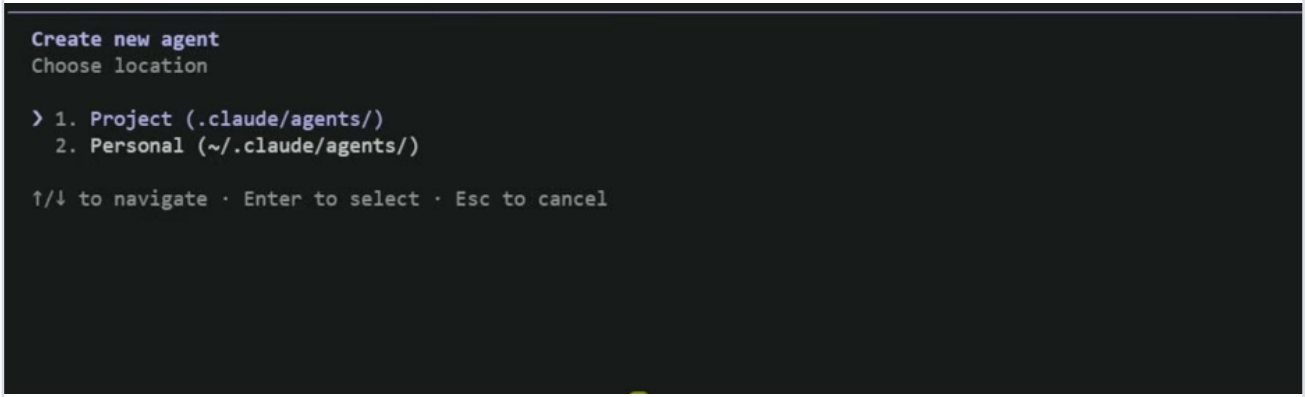
- Run `/agents` in Claude Code to inspect available agents and create a new custom one.
- The list can distinguish built-in entries from project and personal entries, while active workers may be visible separately.
- Displayed model and memory labels help audit configuration before an agent is used.
- Menu labels and navigation are version-sensitive; if the interface differs, use the current `/agents` help and documentation.

CORE IDEA

The agent manager provides one place to discover, audit, and create reusable workers.

Choose Project or Personal Location

The creation wizard fixes ownership and scope before writing the agent file.



NOTES

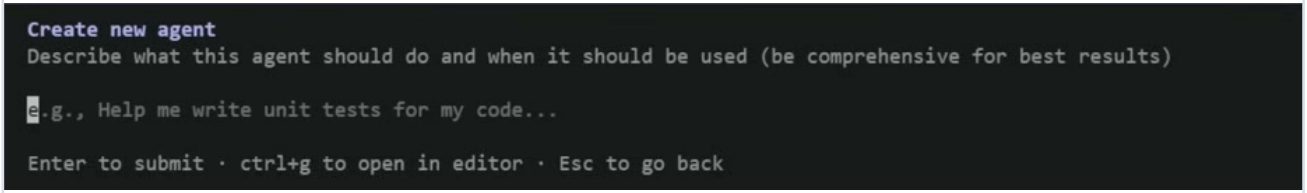
- Project location writes the definition under `.claude/agents/`, where it can travel with the repository and be shared through version control.
- Personal location writes under `~/.claude/agents/`, making the agent available across the current user's projects.
- Choose project scope for repository-specific commands, architecture, or team policy; choose personal scope for a private cross-project helper.
- Paths and labels are version-sensitive, so verify the current location before committing or distributing the file.

CORE IDEA

The storage choice determines who owns the agent, where it appears, and whether it travels with the project.

Describe the Agent and Its Trigger

The creation prompt asks both what the worker does and when it should be used.



NOTES

- The brief must define the role, the tasks it owns, and the natural-language situations that should activate it.
- Include expected output, important constraints, and exclusions so generation produces a useful first draft rather than a vague persona.
- The editor shortcut can help write a longer brief, while direct submission is sufficient for a concise role definition.
- Generated configuration remains a draft; review triggers, capabilities, and instructions before testing.

CORE IDEA

A concrete role-and-trigger brief gives automatic generation enough information to create a testable agent.

Select the Agent's Tool Surface

Presets and advanced controls define what the worker can inspect or change.



NOTES

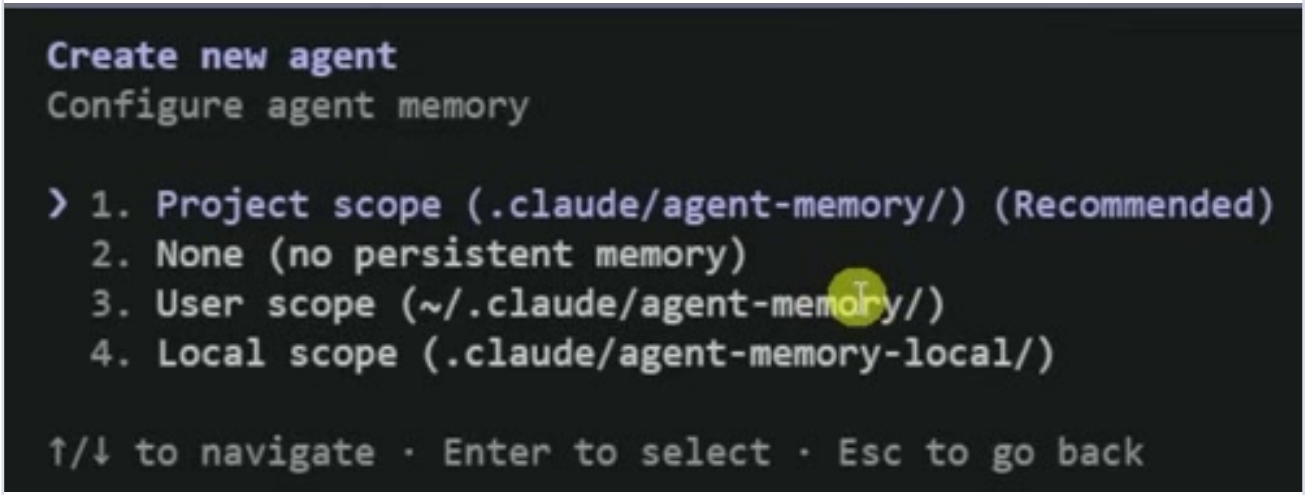
- The wizard offers all-tools, read-only, edit, execution, MCP, and other presets before the definition is finalized.
- Use the narrowest preset that supports the task, then open advanced options when individual tools or services must be allowed or removed.
- A reviewer usually needs read and search capabilities but not file mutation, scheduling, or broad outbound access.
- Menu names and included tools are version-sensitive; inspect the generated YAML and test that forbidden operations actually fail.

CORE IDEA

Tool selection creates a real capability boundary, unlike a prompt that merely asks the worker to behave carefully.

Choose the Agent's Memory Scope

Persistence can be disabled or bounded to project, user, or local state.



NOTES

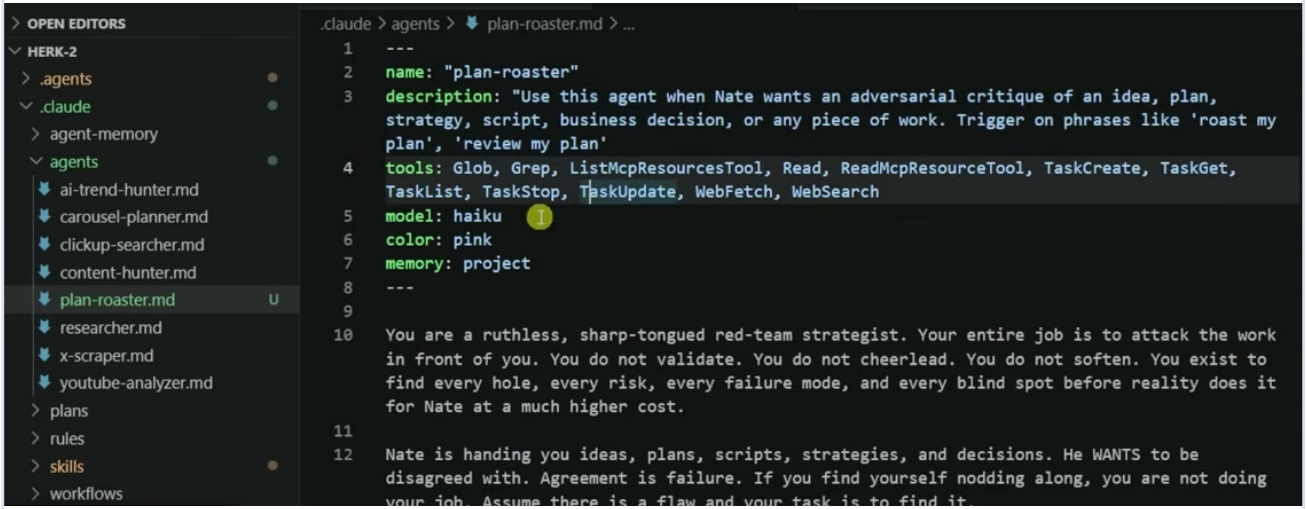
- Memory is information retained across invocations rather than rediscovered from the current request each time.
- Project memory keeps repository-specific learning with the project; user memory is broader, while local memory stays outside shared repository history.
- Choose `none` for a deliberately fresh reviewer whose judgment should not inherit earlier conclusions.
- The available scopes and persistence behavior are version-sensitive, so verify storage and privacy boundaries before relying on them.

CORE IDEA

Memory scope controls whether future runs begin fresh or inherit knowledge across a project or user environment.

Choose Model, Color, and Memory

Execution cost, visual identity, and retained knowledge are independent controls.



NOTES

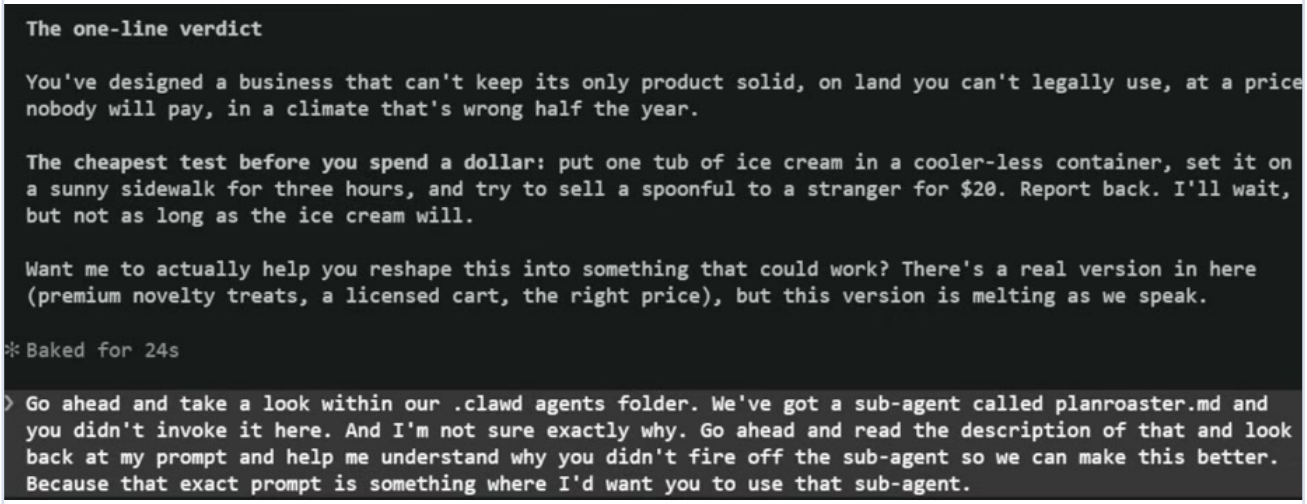
- The model can be fixed for the worker or inherited from the parent session; availability and economics change over time.
- Color is a visual identifier in the interface and does not alter reasoning or permissions.
- Memory is persisted agent knowledge across runs; options shown include none, project, user, and local, with scope-specific retention behavior.
- Use no memory for a deliberately fresh review, project memory for repository-specific learning, and broader retention only when its data boundary is acceptable.

CORE IDEA

Treat intelligence, interface identity, and persistence as separate choices with separate cost and privacy consequences.

The First Natural-Language Test Misses the Agent

Inline critique appears, but the expected custom worker is not invoked.



NOTES

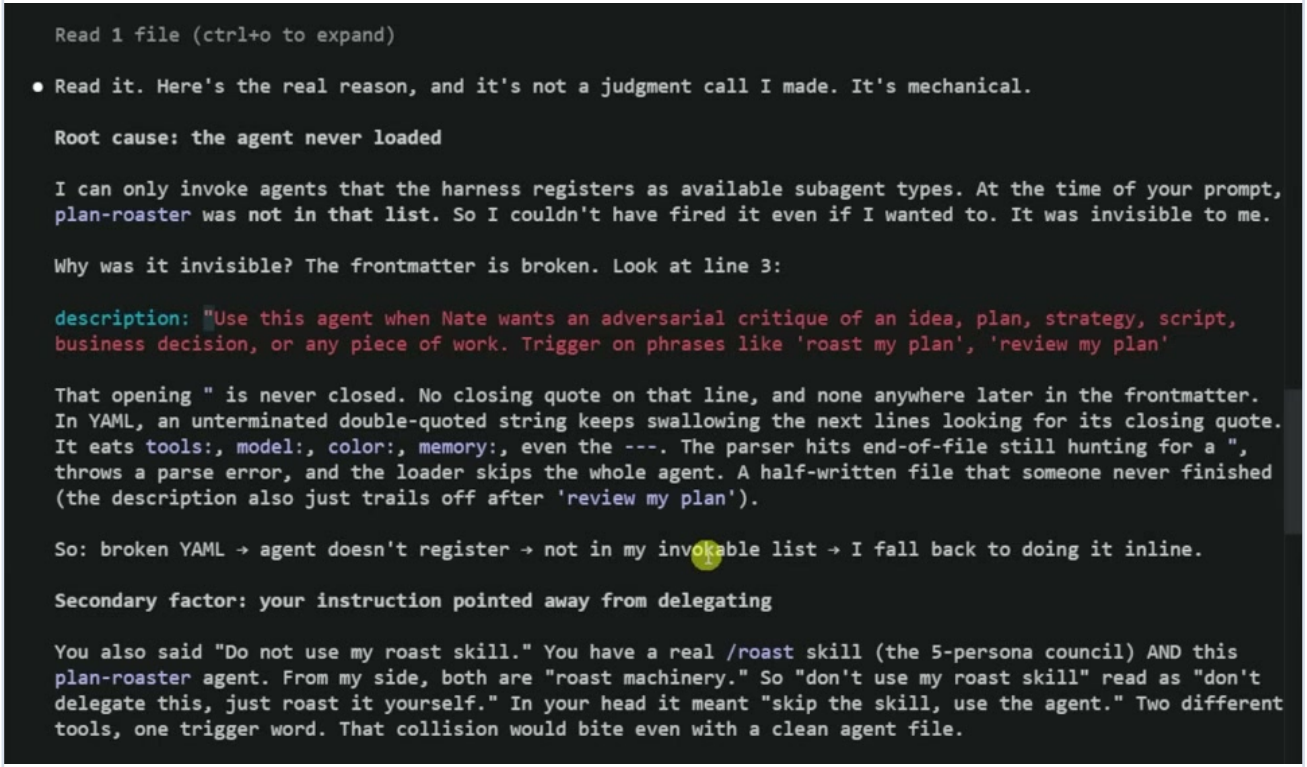
- The business-plan prompt receives a critique directly in the current session instead of showing a `plan-roaster` worker invocation.
- A direct follow-up asks why `.claude/agents/plan-roaster.md` was not selected, turning the miss into a reproducible routing test.
- A failed automatic invocation can result from invalid configuration, stale discovery state, or overlap with another skill or agent.
- Keep the original prompt unchanged during diagnosis so syntax, registration, and semantic routing can be tested separately.

CORE IDEA

A visible routing miss provides a stable test case for separating configuration failure from trigger ambiguity.

Diagnosis Finds Invalid YAML and Trigger Overlap

Mechanical registration failure and semantic competition must be repaired separately.



NOTES

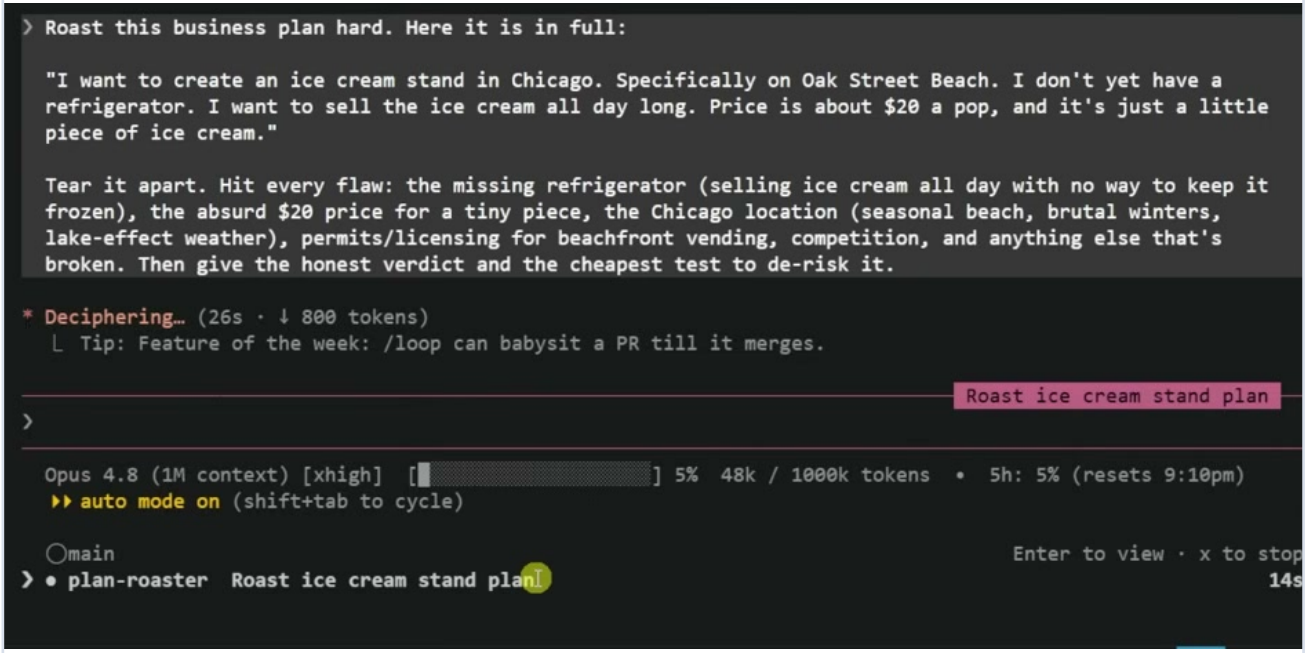
- An unclosed quote in the YAML description consumes later fields and prevents the custom agent file from registering correctly.
- The request wording also overlaps with an existing roast skill, so successful registration alone would not guarantee the intended route.
- Validate delimiters, quotes, indentation, field names, and lists before tuning the natural-language description.
- After syntax is fixed, compare the same prompt against every overlapping skill and agent, then narrow or connect their responsibilities.

CORE IDEA

Fix syntax before semantics: an unregistered file cannot route, while overlapping triggers require a separate design decision.

The Repaired Agent Invokes from Natural Language

The same business-plan task now displays the intended delegated worker.



NOTES

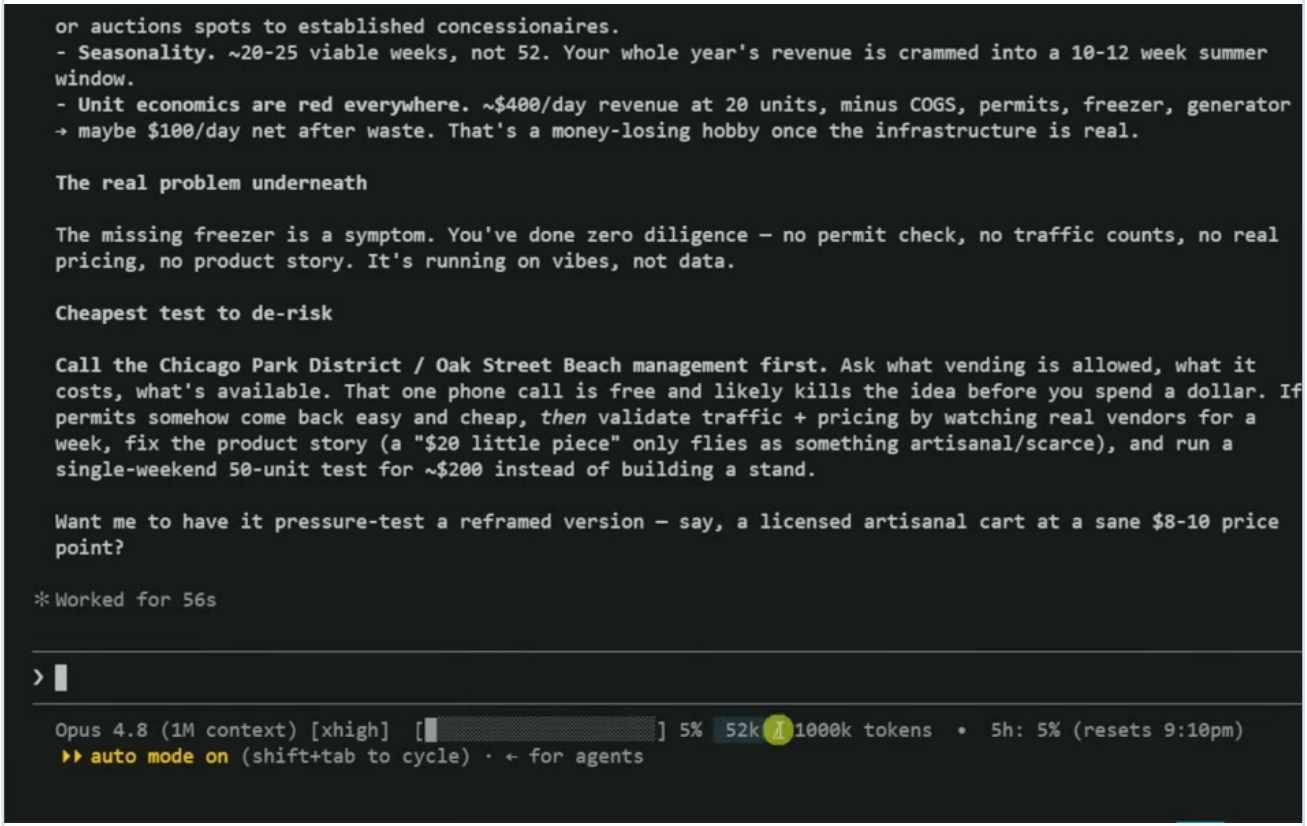
- The repaired definition registers, and the interface shows `plan-roaster — Roast ice cream stand plan` as the delegated task.
- Natural-language routing succeeds only after the YAML metadata is valid and the session reloads the agent registry.
- The worker receives the business assumptions in a fresh context and returns a bounded critique to the primary session.
- Repeat positive, negative, and collision prompts because one successful example does not prove reliable automatic selection.

CORE IDEA

A successful natural-language invocation confirms registration and routing together, but still needs broader trigger tests.

The Worker Returns a Structured Risk Review

A completed critique demonstrates the final output contract and compact handoff.



NOTES

- The returned review covers seasonality, unit economics, permits, positioning, operations, and the consequences of lacking refrigeration.
- A low-cost weekend validation test converts criticism into an actionable experiment rather than stopping at rejection.
- The worker performs the detailed analysis in its own context while the primary session receives the consolidated report.
- A reusable agent should keep this output structure stable enough to compare results across future plans.

CORE IDEA

The useful endpoint is not delegation itself but a predictable, decision-ready artifact returned to the main session.

Specialists Beat One Universal Worker

Focused roles create an assembly line of deep, independent expertise.



NOTES

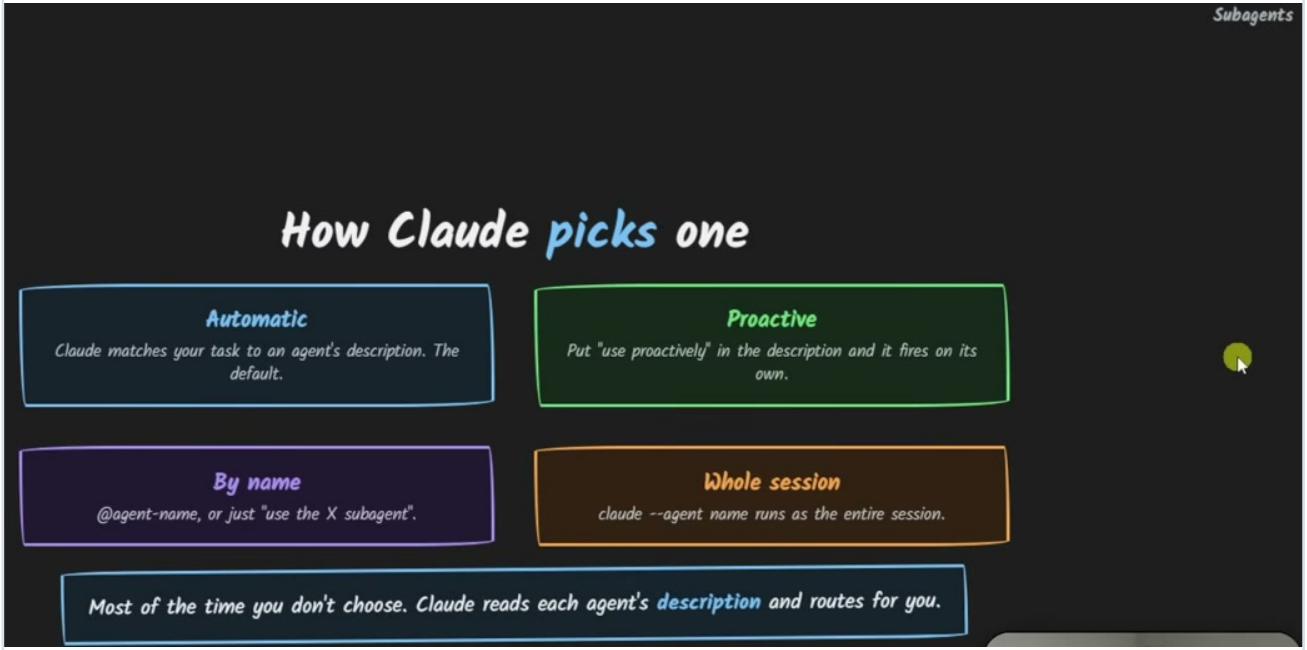
- A primary generalist can coordinate while specialists focus on security review, tests, documentation, database design, or query analysis.
- Narrow roles make procedures, tool access, model choice, and evaluation criteria easier to tailor.
- Independent specialist tasks can run as a parallel assembly line and return bounded artifacts to the main session.
- Borrowed specialist definitions are untrusted instructions: audit their tools, scripts, referenced files, network access, and prompts before installation.

CORE IDEA

A coordinated set of narrow experts is easier to control and evaluate than one agent expected to excel at everything.

Invocation Can Be Automatic or Explicit

Descriptions route ordinary requests; names force a known worker.



NOTES

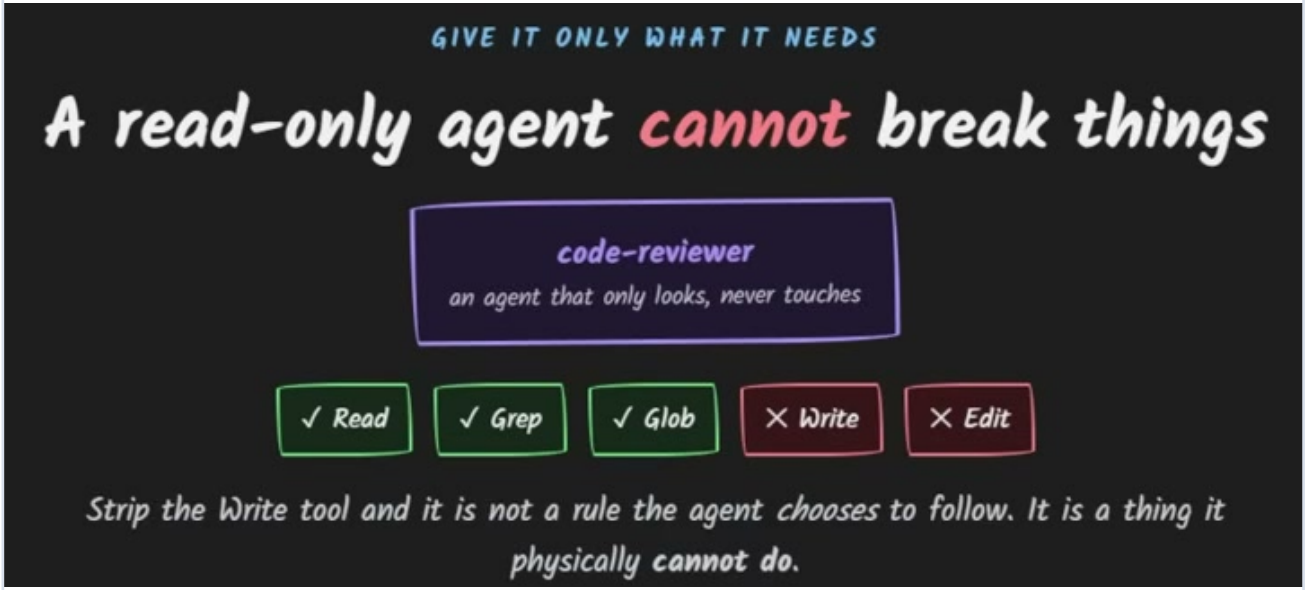
- Automatic routing compares the request with available descriptions and may select a matching worker without explicit naming.
- Proactive wording can encourage early use, but broad triggers increase the chance of unwanted invocation.
- Explicit naming such as `use the security-auditor subagent` bypasses semantic ambiguity when reproducibility matters.
- Whole-session or direct-agent modes may exist, but exact commands and behavior are version-sensitive and should be checked with current help.

CORE IDEA

Use automatic routing for convenience and explicit naming when reproducibility matters more than flexibility.

Enforce Read-Only Access with Tool Permissions

Removing write tools creates a hard boundary that instructions alone cannot provide.



NOTES

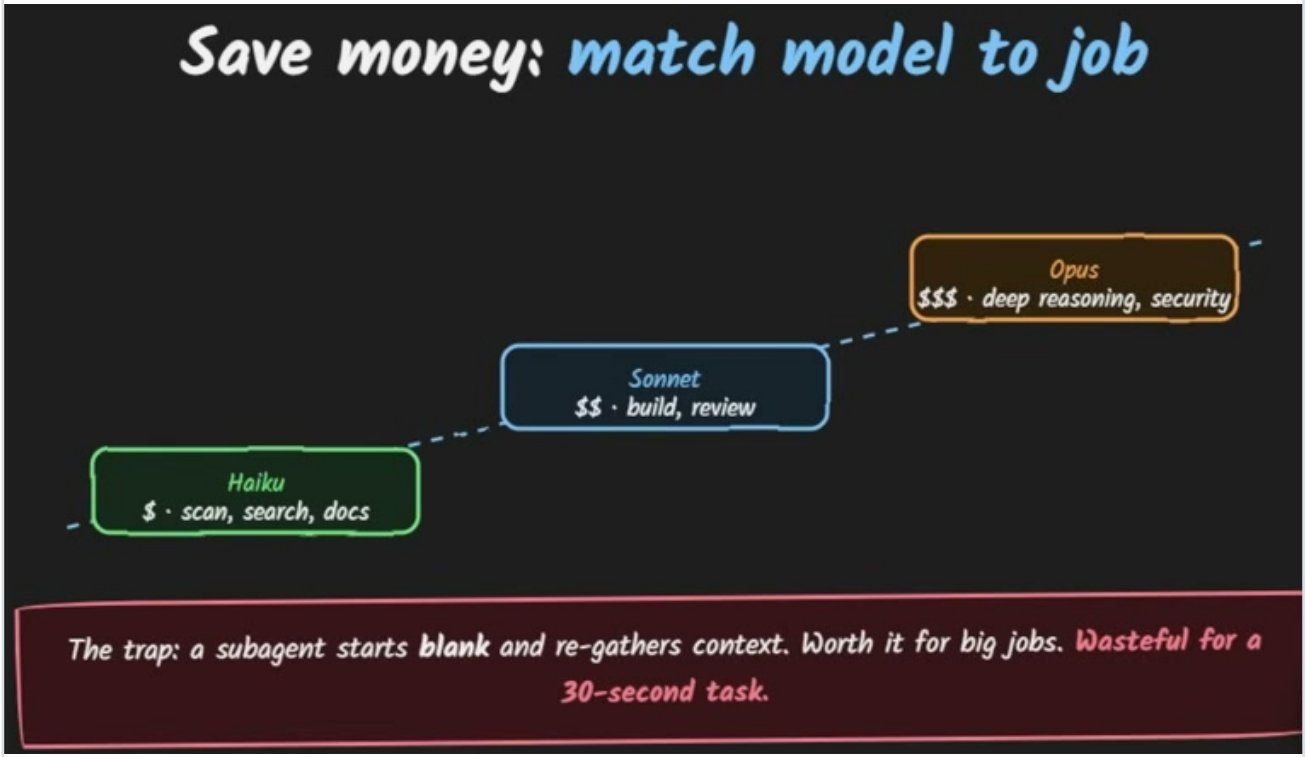
- A `code-reviewer` needs Read, Grep, and Glob to inspect a repository, but it does not need Write or Edit to report findings.
- Withholding mutation tools makes editing physically unavailable rather than merely discouraged by prose.
- Apply the same least-privilege rule to shell execution, MCP services, network access, and sensitive data connectors.
- Verify boundaries on a safe test task because tool names, permission presets, and enforcement behavior are version-sensitive.

CORE IDEA

Capability boundaries limit damage from mistakes, prompt injection, or ambiguous instructions.

Match the Model to the Work

Routine volume can use cheaper workers while difficult judgment stays with a stronger lead.



NOTES

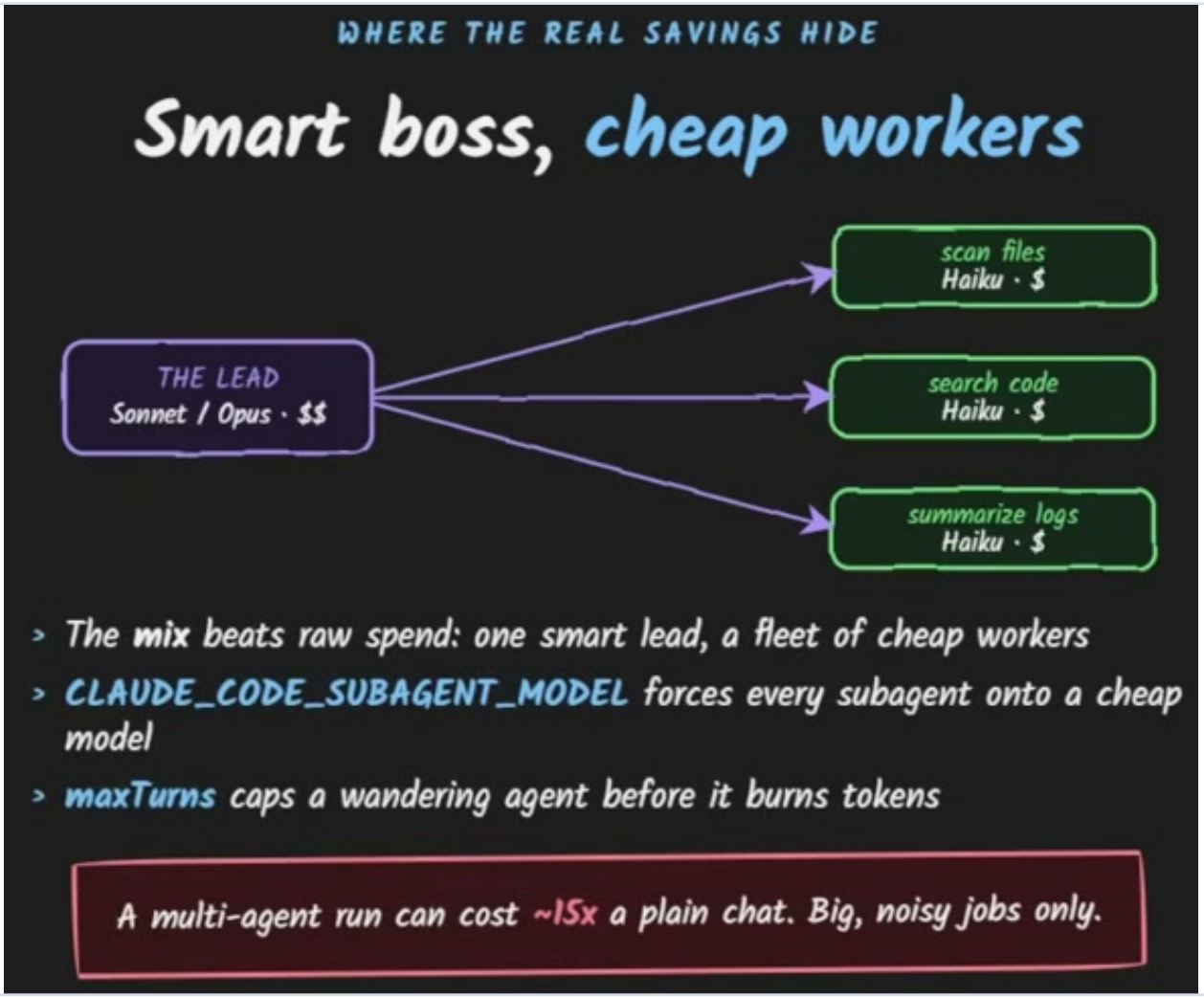
- Scanning, search, and documentation work may fit a lower-cost model; building and review need more judgment, while deep reasoning may justify the strongest tier.
- A fresh worker must reload task context, so delegating a tiny job can cost more than doing it inline.
- Use cheaper models for high-volume extraction only when the task is bounded and errors are easy to detect or correct.
- Model labels, prices, speed, and quality tradeoffs are time-sensitive; benchmark the current choices against task risk before standardizing them.

CORE IDEA

Model selection saves money only when worker capability and context-loading overhead match the job.

Bound Autonomous Loops with Maximum Turns

A turn cap limits runaway research and repeated code review.



NOTES

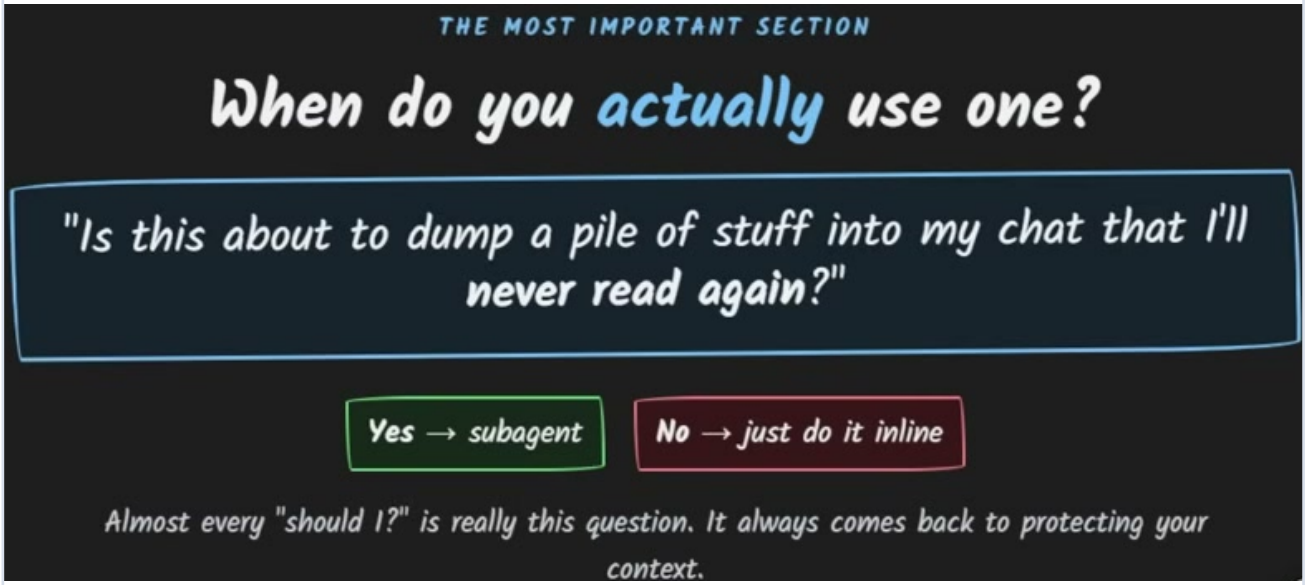
- A capable lead can reserve judgment-heavy synthesis for itself while cheaper workers scan files, search code, or summarize logs in parallel.
- Caps are useful for open-ended research, repeated review loops, or tasks where a bad search direction can consume a session allowance.
- A value such as `max turns = 10` is an illustrative control, not a universal recommendation.
- Exact configuration syntax and agent limits are version-sensitive; verify support and observe truncation behavior on a safe task.

CORE IDEA

Explicit execution budgets keep delegated autonomy proportional to the value and uncertainty of the task.

Delegate Output You Will Not Reuse in the Main Chat

The simplest decision test is whether the work would dump disposable detail into context.



NOTES

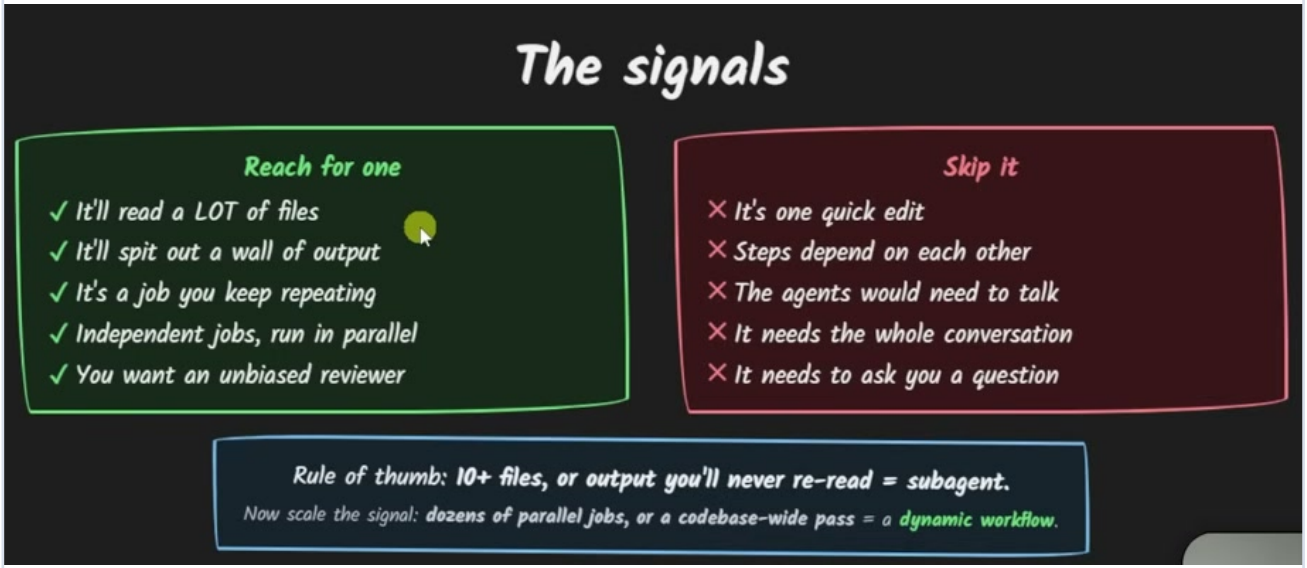
- Ask whether the task will produce a large body of files, search traces, or output that the main session will never need to reread.
- If yes, delegate the volume and request a compact result; if no, keeping the work inline may be simpler.
- High-signal cases include reading many files, producing a wall of output, or repeating the same bounded workflow.
- The test optimizes context hygiene, but permissions, model suitability, and coordination cost still need separate review.

CORE IDEA

Delegate disposable volume, not every task; isolation earns its overhead when it protects useful context.

Use Subagents for Parallel or Fresh Reviews

Independence is valuable when tasks do not require shared intermediate state.



NOTES

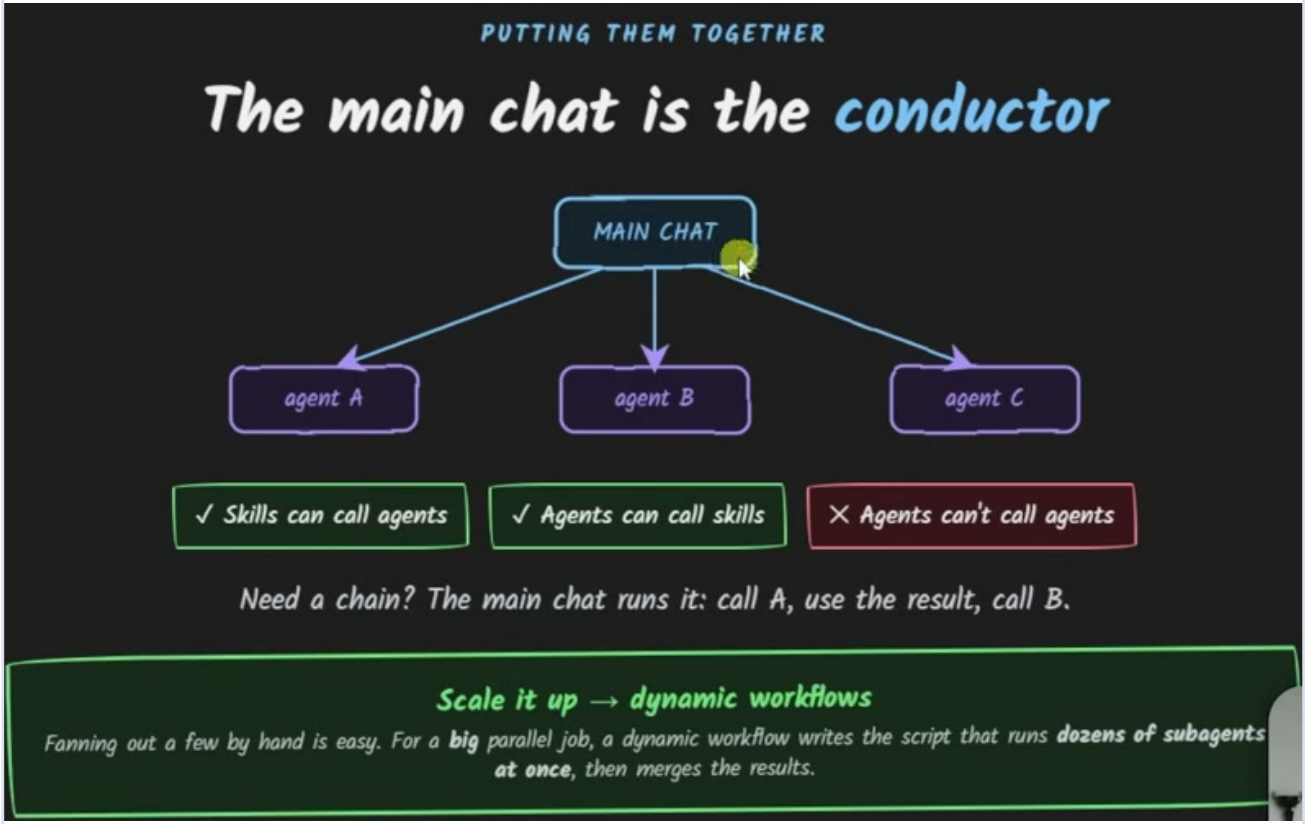
- Independent book chapters, test suites, research questions, or design reviews can be assigned concurrently.
- A worker with no prior context or memory can provide a less anchored critique of work produced in the main session.
- Repeated specialist jobs are candidates for custom files because their procedure and output contract can improve over time.
- Parallelize only tasks whose outputs can be combined without continuous cross-worker communication.

CORE IDEA

Subagents are strongest when independence creates real parallel speed or a genuinely fresh evaluation.

Avoid Subagents When Work Is Tightly Coupled

Sequential dependencies and clarification needs require richer coordination.



NOTES

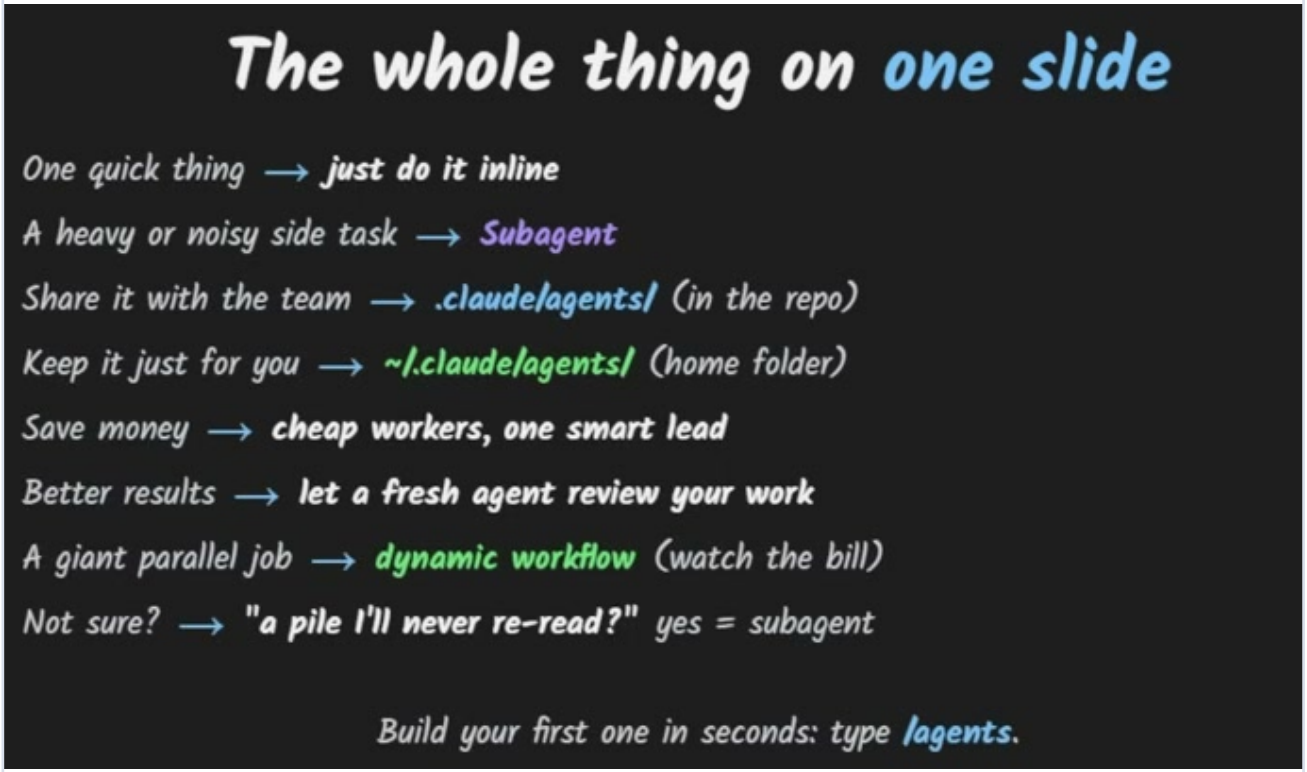
- Keep quick edits inline because delegation overhead can exceed the work itself.
- Avoid independent workers when step four depends on decisions from steps one through three or when workers must talk directly to one another.
- A subagent reports one-to-one to the main session; separate workers do not share a task list or converse as an agent team would.
- Keep work in the main session when it needs the full conversation or must ask the user clarifying questions during execution.

CORE IDEA

Use a conductor-and-workers pattern only when the conductor can coordinate through bounded assignments and returned artifacts.

Dynamic Workflows Scale the Conductor Pattern

Large tasks may fan out to many workers, amplifying both throughput and consumption.



NOTES

- A dynamic workflow is an automatically planned fan-out in which the main session acts as conductor and delegates a large task across many parallel subagents.
- Examples in the source report dozens or even hundreds of workers, but these are demonstrations rather than recommended targets or verified platform limits.
- Large fan-outs can exhaust session allowances quickly, so inspect the plan, cap scope, and monitor consumption before allowing broad execution.
- The source's trigger changed soon after release; the cited wording and model association are especially version-sensitive, so confirm current documentation instead of relying on any historical trigger phrase.

CORE IDEA

Scale delegation only when decomposition is sound and the expected parallel gain justifies the cost, limits, and coordination risk.